Sakey keys and Keys Graph for Entity Alignment

Ilyes SAIS `ilyes.sais@universite-paris-saclay.fr`
Wenchong PAN `wenchong.pan@universite-paris-saclay.fr`

March 4, 2026

1 Introduction

Entity alignment is the task of detecting when two RDF resources from different knowledge graphs (KGs) refer to the same real-world entity. In the context of Linked Data, these identity links are typically represented using `owl:sameAs`. Correct alignment is a fundamental step for data integration, knowledge graph completion, and cross-source querying. When entities originating from multiple sources are correctly aligned, information from different knowledge graphs can be integrated and queried in a unified manner, enabling richer knowledge representation and more accurate reasoning.

A classical approach to entity alignment relies on the notion of *keys*. Intuitively, a key is a set of properties whose values uniquely identify an entity. However, RDF data is often incomplete and heterogeneous, meaning that strict keys rarely exist in practice. Many entities may miss certain attribute values, and different datasets may describe the same entity using different modeling choices. For this reason, approaches based on *almost-keys* have been proposed. An almost-key is a set of properties that uniquely identifies entities in most cases, allowing a small number of violations due to noise or missing values in the data.

Graph keys extend the traditional concept of keys by incorporating *property paths*. Instead of relying only on direct attributes of an entity, graph keys may also consider information accessible through neighboring nodes in the RDF graph. For example, an entity may be identified not only by its own properties but also by properties of related entities reached through object properties. This contextual information can significantly increase the discriminative power of keys in knowledge graphs.

The objective of this project is therefore to study the use of keys and graph keys for entity alignment in RDF datasets. First, candidate almost-keys are discovered using the SAKey tool. These keys are then used to generate identity links between entities of two RDF datasets. In a second step, the discovered keys are extended to graph keys by incorporating property paths that capture contextual information in the graph structure. Finally, the quality of the predicted identity links is evaluated using a gold standard alignment provided with the dataset.

To analyze the impact of different matching strategies, three approaches are considered in this study. The first approach, referred to as *SAKey Baseline*, directly applies the discovered SAKey keys to generate identity links without any additional filtering or blocking mechanism. The second approach, *SAKey Optimized*, improves the matching process by introducing typed blocking and empty-signature filtering, which significantly reduces the number of false positive matches. The third approach, *Key Graphs*, further extends the key-based strategy by constructing graph keys using property paths and contextual information from neighboring nodes. The experimental comparison of these three methods allows us to better understand the trade-offs between precision, recall, and the expressiveness of key-based entity alignment methods.

2 Dataset Description

We use the **SPIMBENCH** small dataset, a benchmark designed for evaluating entity matching and data integration techniques over RDF knowledge graphs. The dataset contains two RDF graphs representing two heterogeneous data sources: `Abox1.nt` and `Abox2.nt`. Each file contains a set of RDF triples serialized in N-Triples format.

Formally, an RDF graph is defined as a set of triples:

$$(s, p, o) \in U \times U \times (U \cup L \cup B)$$

where U denotes the set of URI resources, L denotes the set of literals, and B denotes the set of blank nodes. Each triple represents a statement in which a subject s is related to an object o through a predicate p.

In our dataset, entities correspond to URI resources appearing as subjects in the RDF triples. These entities are described through a variety of properties such as labels, descriptions, or contextual relations to other entities. Objects may therefore be:

- URI resources referring to other entities,
- literal values such as strings or dates,
- blank nodes representing anonymous resources.

The two graphs `Abox1.nt` and `Abox2.nt` represent two different descriptions of the same underlying real-world entities. Although they describe overlapping information, the graphs differ in terms of structure, available attributes, and completeness. This heterogeneity makes entity alignment a non-trivial task, since identical entities may be described using different properties or missing values.

For an entity e and predicate p, we denote by

$$V(e,p) = \{o \mid (e,p,o) \text{ is a triple in the graph}\}$$

the set of values associated with property p for entity e. These property values are used to construct the *signature* of an entity under a given key or graph key.

In addition to the two RDF graphs, the dataset provides a reference alignment file `refalign.rdf`. This file contains the ground-truth identity links between entities appearing in the two graphs. Each link represents a pair of entities that correspond to the same real-world object and is typically expressed using the `owl:sameAs` relation.

Let

$$G = \{(a,b) \mid a \in \text{Abox1},\ b \in \text{Abox2},\ a \equiv b\}$$

denote the set of gold standard alignment pairs extracted from `refalign.rdf`. Each pair $(a,b) \in G$ indicates that the entities a and b refer to the same real-world entity.

The gold alignment plays a crucial role in the evaluation of entity matching methods. Given a set of predicted identity links $\widehat{G}$ produced by a matching algorithm, the quality of the alignment can be assessed by comparing $\widehat{G}$ with the gold standard set G.

In the SPIMBENCH small dataset used in this project, the number of gold identity links is:

$$|G| = 299.$$

These links provide a reliable ground truth for evaluating the precision, recall, and F1-score of the proposed matching approaches. The dataset therefore allows us to quantitatively analyze how well different key-based strategies, including SAKey baseline, SAKey optimized, and graph keys are able to recover correct identity links between the two RDF graphs.

3 Methodology

3.1 SAKey Baseline

SAKey takes an RDF dataset and discovers sets of properties that behave as keys or almost-keys for a given class (or globally depending on the setting). In practice, we use the `-1-almost keys` produced by SAKey as candidate keys.

A classical key is a set of predicates:

$$K = \{p_1, p_2, \ldots, p_k\}.$$

For an entity e, we define its (multi-valued) *signature* under key K as:

$$\text{sig}_K(e) = \big(V(e,p_1), V(e,p_2), \ldots, V(e,p_k)\big),$$

where $V(e,p)$ is the sorted tuple of all objects o such that (e,p,o) is an RDF triple.

SAKey Key Discovery Algorithm

SAKey discovers candidate keys by exploring combinations of RDF predicates and verifying whether they uniquely identify entities in the dataset. The algorithm follows a search strategy similar to frequent itemset mining, where candidate predicate sets are gradually expanded and evaluated.

Let $P = \{p_1, p_2, \ldots, p_m\}$ denote the set of predicates appearing in the RDF graph. The goal is to find subsets of predicates $K \subseteq P$ that uniquely identify entities.

For a candidate predicate set K, SAKey evaluates whether the following condition holds:

$$\forall e_1, e_2,\ \mathrm{sig}_K(e_1) = \mathrm{sig}_K(e_2) \Rightarrow e_1 = e_2$$

If the condition is satisfied, K is considered a valid key. We bucket entities by signature and then output all cross-products between buckets with the same signature.

However, due to missing values and noise in RDF data, strict keys rarely exist. Therefore SAKey introduces the notion of *almost-keys*. A predicate set K is considered an almost-key if the number of violations is below a threshold ϵ:

$$|\{(e_1, e_2) \mid \mathrm{sig}_K(e_1) = \mathrm{sig}_K(e_2),\ e_1 \neq e_2\}| \leq \epsilon$$

In this project we use the `-1-almost keys` produced by SAKey, which allow a small number of violations while still providing strong identification power.

SAKey explores the predicate space using a level-wise search strategy. The main steps of the algorithm can be summarized as follows:

Algorithm 1 SAKey key / almost-key discovery

Require: RDF dataset D; predicate set P extracted from D; violation threshold ϵ
Ensure: Set of candidate (almost-)keys $\mathcal{K}$

```
K ← ∅
C_1 ← {{p} | p ∈ P}                                   ▷ size-1 candidates
ℓ ← 1
while C_ℓ ≠ ∅ do
    for all K ∈ C_ℓ do
        Compute signatures {sig_K(e)}_{e∈Ent(D)}
        v ← #{(e_1, e_2) | sig_K(e_1) = sig_K(e_2), e_1 ≠ e_2}   ▷ violations
        if v ≤ ε then
            K ← K ∪ {K}                                   ▷ K is an almost-key
        else
            Mark K as invalid (used for pruning supersets)
        end if
    end for
    C_{ℓ+1} ← JoinAndPrune(C_ℓ)
    ℓ ← ℓ + 1
end while
return K
```

To avoid exploring the entire exponential search space, SAKey applies pruning rules based on key monotonicity properties. If a predicate set already violates the almost-key condition, all its supersets can be safely pruned from the search.

The output of SAKey is therefore a collection of candidate predicate sets that can potentially identify entities. These candidate keys are later used to generate identity links between entities from different RDF graphs.

3.1.1 baseline Limitation.

RDF data often contains missing property values. If $V(e, p) = \emptyset$ for many entities, then many signatures become identical, producing huge buckets and thus huge numbers of predicted links. This yields very high recall but extremely low precision.

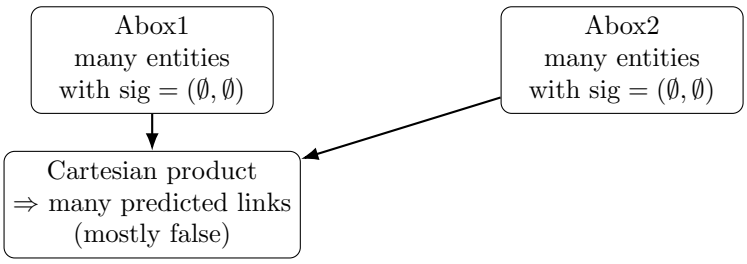


Figure 1: SAKey Baseline Problem

3.2 SAKey Optimized

The SAKey Optimized approach builds upon the baseline matching procedure but introduces additional filtering mechanisms in order to reduce the large number of false positives produced by the baseline. While the baseline compares entities solely based on their key signatures, the optimized approach additionally enforces type compatibility and removes entities with empty signatures.

The objective is to reduce the number of meaningless comparisons while preserving the ability to identify correct matches.

Let $\mathcal{C}(e)$ denote the set of RDF types of entity e. The optimized matching rule therefore becomes:

$$(e_1, e_2) \in \widehat{G}_K^{\text{opt}} \iff \Big(\text{sig}_K(e_1) = \text{sig}_K(e_2)\Big) \wedge \Big(\mathcal{C}(e_1) \cap \mathcal{C}(e_2) \neq \emptyset\Big) \wedge \Big(\text{sig}_K(e_1) \neq \emptyset \wedge \text{sig}_K(e_2) \neq \emptyset\Big).$$

This rule keeps the signature equality condition of the baseline method, but adds two additional constraints: type compatibility and non-empty signatures.

(1) Typed blocking

Entities in RDF graphs are often annotated using the predicate `rdf:type`. We exploit this information to restrict comparisons to entities that share at least one common semantic type.

Let $\mathcal{C}(e)$ denote the set of RDF classes associated with entity e. Two entities are considered comparable only if they share at least one type:

$$(e_1, e_2) \text{ is comparable} \iff \exists c \,:\, (e_1, \text{rdf:type}, c) \;\wedge\; (e_2, \text{rdf:type}, c)$$

or equivalently:

$$\mathcal{C}(e_1) \cap \mathcal{C}(e_2) \neq \emptyset$$

This mechanism prevents comparisons between semantically incompatible entities (for example a `Person` and a `CreativeWork`), thereby reducing many obvious false matches.

(2) Empty-signature filtering

Another major issue observed in the baseline method comes from entities whose signatures contain no information.

Recall that the signature of an entity under key $K = \{p_1, \ldots, p_k\}$ is

$$\text{sig}_K(e) = \big(V(e, p_1), V(e, p_2), \ldots, V(e, p_k)\big)$$

where $V(e, p)$ denotes the set of values associated with predicate p.

If an entity has no values for any predicate in the key, its signature becomes:

$$\text{sig}_K(e) = (\emptyset, \ldots, \emptyset)$$

Such entities provide no discriminative information and tend to generate very large buckets during signature grouping. To avoid the resulting combinatorial explosion of candidate matches, we simply discard entities whose signatures are entirely empty:

$$\text{sig}_K(e) = (\emptyset, \ldots, \emptyset) \quad \Rightarrow \quad e \text{ is ignored for matching.}$$

Algorithm 2 SAKey Optimized

Require: Entities E_1 from Abox1, entities E_2 from Abox2, candidate key K
Ensure: Predicted identity links $\widehat{G}_K^{opt}$

```
1: Ĝ_K^opt ← ∅
2: for all e ∈ E_1 ∪ E_2 do
3:     Compute signature sig_K(e)
4: end for
5: Remove entities with empty signatures
```

$$\mathrm{sig}_K(e) = (\emptyset, \ldots, \emptyset)$$

```
6: Group remaining entities into buckets according to their signatures
7: for all signature value s appearing in both graphs do
8:     for all e_1 ∈ bucket_1(s) do
9:         for all e_2 ∈ bucket_2(s) do
10:            if C(e_1) ∩ C(e_2) ≠ ∅ then
11:                Ĝ_K^opt ← Ĝ_K^opt ∪ {(e_1, e_2)}
12:            end if
13:        end for
14:    end for
15: end for
16: return Ĝ_K^opt
```

3.3 Key Graphs

A graph key extends classical keys by including *paths* in the RDF graph. Instead of using only direct attributes of an entity, graph keys allow information to be collected from neighboring nodes.

We represent a path of length two as $(p \parallel q)$, meaning that we first follow an object property p to reach a neighbor node, and then read property q from that neighbor.

A graph key therefore consists of both direct predicates and path-based predicates:

$$GK = \{p_1, \ldots, p_r,\ (p_{r+1} \parallel q_{r+1}), \ldots, (p_k \parallel q_k)\}.$$

Here:

- $p_1, \ldots, p_r$ are direct predicates of the entity,
- $(p \parallel q)$ denotes a property path feature.

Graph keys capture both local attributes and contextual information from the RDF graph structure.

Graph signature

For direct predicates the definition remains identical to the classical key case. Let $V(e, p)$ denote the set of values of predicate p for entity e.

$$V(e, p) = \{o \mid (e, p, o) \in G\}$$

For a path $(p \parallel q)$ we first follow predicate p to reach neighbor nodes and then collect their q values:

$$V(e, p \parallel q) = \bigcup_{x:(e,p,x)} V(x, q).$$

The graph signature of entity e under graph key GK is therefore defined as

$$\mathrm{sig}_{GK}(e) = \big(V(e, p_1), \ldots, V(e, p_r), V(e, p_{r+1} \parallel q_{r+1}), \ldots, V(e, p_k \parallel q_k)\big).$$

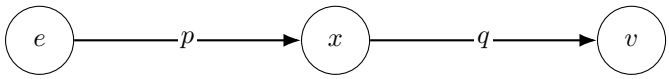
This signature integrates both direct attributes and contextual features extracted from the neighborhood of the entity.

Intuition and depth limitation

Graph keys can be more discriminative than classical keys because they incorporate contextual information. For example, a person may not be uniquely identified by a single attribute, but the combination of attributes of related entities (e.g., affiliations, organizations, or locations) can help distinguish it.

However, longer paths also increase the probability of missing values. If intermediate nodes lack the required properties, the resulting signatures may become incomplete.

For this reason, in this project we restrict graph keys to a *depth-1 expansion*, corresponding to paths of length two ($p \parallel q$). This allows us to capture useful contextual information while limiting the propagation of missing values.



$$V(e, p \parallel q) = \bigcup_{x:(e,p,x)} V(x, q)$$

Figure 2: GraphKey path.

Graph-based matching rule

Using graph keys, two entities are predicted to represent the same real-world entity if their graph signatures are identical:

$$(e_1, e_2) \in \widehat{G}^{GK} \quad \Longleftrightarrow \quad \mathrm{sig}_{GK}(e_1) = \mathrm{sig}_{GK}(e_2).$$

As in the previous methods, entities are grouped into buckets according to their signatures and candidate links are generated as cross-products of entities sharing the same signature.

Example Consider the following simplified RDF fragments extracted from two datasets.

Subject	Predicate	Object
e_1	worksFor	x_1
x_1	name	"BBC"
e_2	worksFor	x_2
x_2	name	"BBC"

Assume the graph key contains the path predicate:

$$GK = \{worksFor \parallel name\}$$

The graph signature for entity e_1 becomes:

$$V(e_1, worksFor \parallel name) = \bigcup_{x:(e_1,worksFor,x)} V(x, name) = \{\text{"BBC"}\}$$

Similarly, for entity e_2:

$$V(e_2, worksFor \parallel name) = \{\text{"BBC"}\}$$

Therefore the graph signatures become:

$$\mathrm{sig}_{GK}(e_1) = (\{\text{"BBC"}\})$$

$$\mathrm{sig}_{GK}(e_2) = (\{\text{"BBC"}\})$$

Since the signatures are identical, the algorithm predicts the identity link:

$$(e_1, e_2) \in \widehat{G}^{GK}.$$

This example illustrates how graph keys exploit contextual information from neighboring nodes in order to identify entities that might not be distinguishable using only their direct attributes.

Algorithm 3 GraphKey-based Entity Matching

Require: Entities E_1 from Abox1, entities E_2 from Abox2, graph key GK
Ensure: Predicted identity links $\widehat{G}^{GK}$

```
1: Ĝ^GK ← ∅
2: for all e ∈ E1 ∪ E2 do
3:     for all direct predicate p in GK do
4:         Compute V(e, p)
5:     end for
6:     for all path predicate (p ‖ q) in GK do
7:         V(e, p ‖ q) ← ∅
8:         for all x such that (e, p, x) do
9:             V(e, p ‖ q) ← V(e, p ‖ q) ∪ V(x, q)
10:        end for
11:    end for
12:    Construct graph signature sig_GK(e)
13: end for
14: Group entities into buckets according to sig_GK
15: for all signature s appearing in both graphs do
16:    for all e1 ∈ bucket1(s) do
17:        for all e2 ∈ bucket2(s) do
18:            Ĝ^GK ← Ĝ^GK ∪ {(e1, e2)}
19:        end for
20:    end for
21: end for
22: return Ĝ^GK
```

4 Evaluation Method

We evaluate the predicted links against the gold standard G from `refalign.rdf`. Let $\widehat{G}$ be the predicted links for a given method.

$$TP = |\widehat{G} \cap G|, \qquad FP = |\widehat{G} \setminus G|, \qquad FN = |G \setminus \widehat{G}|.$$

The standard evaluation metrics are:

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}, \qquad F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

In knowledge graphs, false positive links (`sameAs` errors) can propagate incorrect facts across merged graphs. This "error propagation" can corrupt downstream tasks such as querying, reasoning, and recommendation. Therefore, precision is often prioritized in practice, especially for automatic integration pipelines.

5 Experimental Results

We report results for three methods: **SAKey Baseline**, **SAKey Optimized**, and **GraphKey**. All evaluation metrics are computed with respect to the same gold standard alignment extracted from `refalign.rdf`, which contains:

$$|G| = 299$$

true identity links between entities from Abox1 and Abox2.

For each method we compute the set of predicted identity links $\widehat{G}$ and evaluate it using Precision, Recall and F1-score.

5.1 SAKey Baseline Results

The SAKey baseline performs entity matching using the discovered candidate keys without any additional filtering or blocking mechanism.

In practice, RDF datasets often contain many missing property values. When an entity does not contain values for the predicates in the key, its signature becomes empty:

$$\mathrm{sig}_K(e) = (\emptyset, \ldots, \emptyset).$$

As a consequence, many entities share the same empty signature and are grouped into the same signature bucket. When matching entities across datasets, this produces a large Cartesian product between buckets.

In our experiments, the best baseline key produces:

$$|\widehat{G}| = 167{,}319, \quad TP = 299, \quad FP = 167{,}020, \quad FN = 0.$$

Hence:

$$Precision = 0.002, \quad Recall = 1.000, \quad F1 = 0.004.$$

This result illustrates the typical behavior of naive key-based matching. Since almost all entities share weak or empty signatures, the baseline approach predicts a huge number of candidate links. While this guarantees that all true matches are discovered (perfect recall), the vast majority of predicted links are incorrect, resulting in extremely low precision.

5.2 SAKey Optimized Results

The SAKey Optimized method improves the baseline pipeline by introducing two filtering mechanisms:

- **Typed blocking**: comparisons are restricted to entities sharing at least one RDF type.
- **Empty-signature filtering**: entities with completely empty signatures are excluded from the matching process.

These two modifications significantly reduce the number of meaningless comparisons between entities.

Using exactly the same candidate keys discovered by SAKey, the optimized matching procedure produces:

$$|\widehat{G}| = 196, \quad TP = 191, \quad FP = 5, \quad FN = 108,$$

Hence:

$$Precision = 0.974, \quad Recall = 0.639, \quad F1 = 0.772.$$

Compared with the baseline approach, the number of predicted links decreases drastically from more than 167 000 to only 196. At the same time, almost all false positives are eliminated, resulting in a very high precision of 97.4%.

Although recall decreases compared with the baseline, the optimized approach still identifies a majority of the true matches, which leads to a strong overall F1-score.

5.3 GraphKey Results

GraphKey extends classical keys by incorporating path-based features. Instead of relying only on direct attributes of an entity, graph keys also exploit contextual information from neighboring nodes.

This additional contextual information often makes signatures more discriminative and therefore reduces ambiguous matches.

Using the same filtering mechanisms (typed blocking and empty-signature filtering), the best GraphKey configuration produces:

$$|\widehat{G}| = 124, \quad TP = 121, \quad FP = 3, \quad FN = 178,$$

Hence:

$$Precision = 0.976, \quad Recall = 0.405, \quad F1 = 0.572.$$

GraphKey achieves the highest precision among all methods. However, recall decreases because path-based signatures require additional property values along the graph paths. If intermediate nodes lack the required information, the signature becomes incomplete and the corresponding entity cannot be matched.

6 Comparison

The comparison clearly illustrates the trade-offs between the three approaches.

The **SAKey Baseline** produces extremely high recall but almost no precision. Because it performs no filtering, it generates an enormous number of candidate links, most of which are incorrect. Such behavior makes the baseline approach impractical for real-world entity alignment tasks.

The **SAKey Optimized** approach significantly improves the situation. By restricting comparisons using semantic types and removing empty signatures, it drastically reduces the number of predicted links while maintaining a high precision. Among the three methods, SAKey Optimized achieves the best overall balance between precision and recall, as reflected by the highest F1-score.

The **GraphKey** method achieves the highest precision. This confirms that path-based contextual information can provide very strong identification signals. However, the use of path features also introduces additional missing values, which reduces recall. Consequently, fewer true matches are discovered.

From an entity alignment perspective, **precision is particularly important**. Incorrect identity links can propagate errors across knowledge graphs and may severely affect downstream applications such as knowledge integration or reasoning. For this reason, high-precision methods such as GraphKey are often preferable when the goal is to produce reliable identity links.

Nevertheless, the experimental results suggest that SAKey Optimized provides the best trade-off between precision and recall in our experiments, achieving both high accuracy and reasonable coverage of the true alignments.

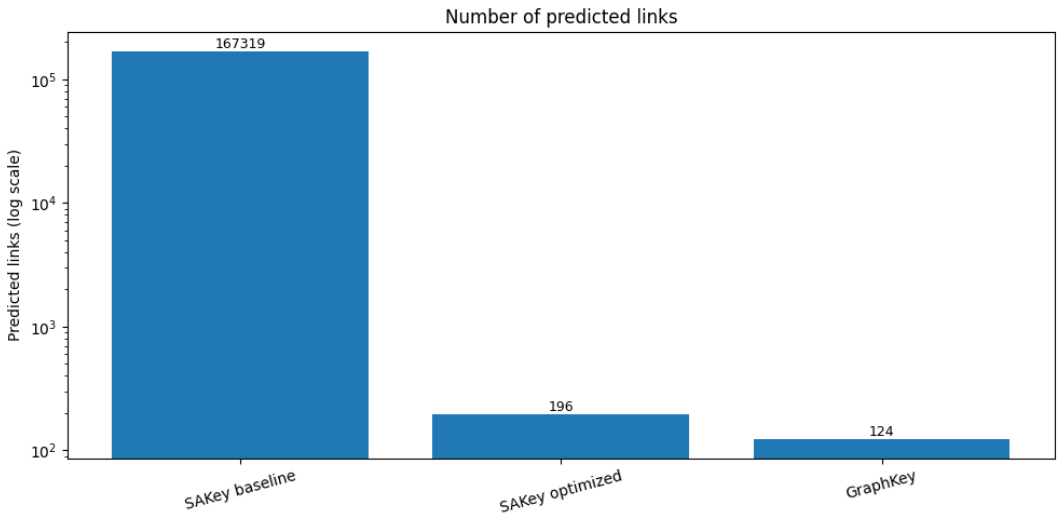


Figure 3: Number of Predicated Links

Table 1: Evaluation metrics for the three methods

Method	$\lvert\widehat{G}\rvert$	Precision	Recall	F1	TP	FP	FN
SAKey Baseline	167,319	0.002	1.000	0.004	299	167,020	0
SAKey Optimized	196	0.974	0.639	0.772	191	5	108
GraphKey	124	0.976	0.405	0.572	121	3	178

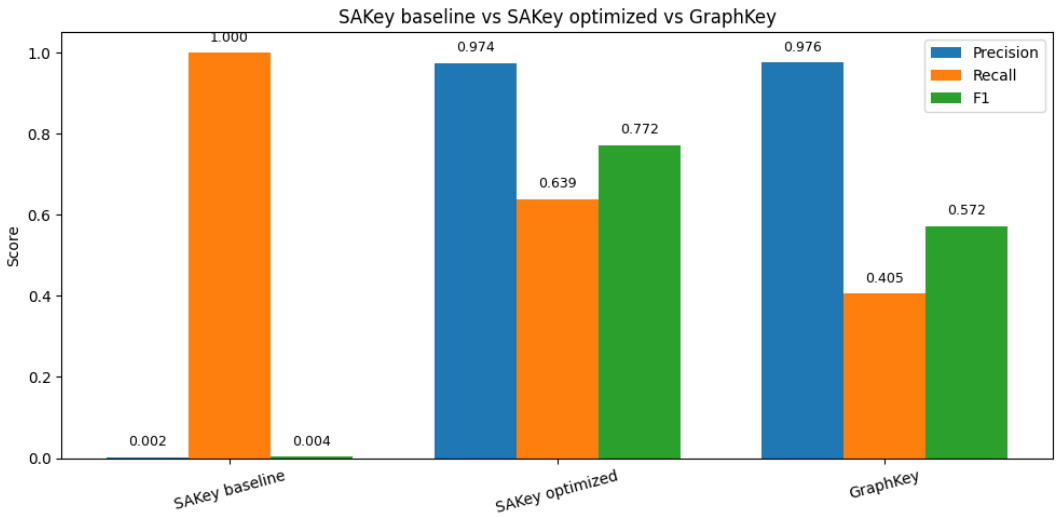


Figure 4: Evaluation

7 Conclusion

The experimental study highlights several key observations regarding key-based entity alignment methods:

- **SAKey Baseline limitations.** The SAKey Baseline achieves perfect recall because it produces a very large number of candidate links. However, weak keys and missing values lead to large signature buckets, generating many false positives. As a result, precision becomes extremely low.
- **Effectiveness of SAKey Optimized.** By introducing *typed blocking* and *empty-signature filtering*, SAKey Optimized significantly reduces the number of meaningless comparisons. This dramatically improves precision (from 0.002 to 0.974) while maintaining a reasonable recall.
- **GraphKey precision advantage.** GraphKey extends classical keys by incorporating contextual information through path-based signatures. This additional structural information makes entity signatures more discriminative, resulting in the highest precision among the tested methods.
- **Recall reduction in GraphKey.** Because GraphKey relies on path-based features, missing values along the paths may prevent the construction of complete signatures. Consequently, some true matches are not detected, leading to a lower recall compared with SAKey Optimized.
- **Importance of precision.** In knowledge graph integration, false positive `owl:sameAs` links can merge unrelated entities and propagate incorrect information across datasets. Therefore, high precision is often more critical than high recall in practical alignment systems.